EgoQuant: the on-chain seal

Status: 20 October 2026. One short document for the judges: what is sealed, where it goes, what the seal proves and what it does not, and how to check it yourself in two minutes. The decisions behind it are recorded by Aslan in `docs/ONCHAIN_DECISIONS.md`; the technical detail is in `docs/ANCHORING.md`; the click-by-click path is `docs/JUDGE_GUIDE.md`.

1. In plain words

When a signed-in investor confirms a plan that the Risk Engine approved, EgoQuant produces the exact report once (its first line says "Sealed for: <username>") and keeps a record of the advice. It then publishes **only a fingerprint** of that record (32 bytes, with a short label) on a public **test** blockchain (Ethereum Sepolia). The investor downloads one file, the **sealed report**. Later anyone can load that file on **Verify a report** (no account needed). The page recomputes the fingerprint in the browser, reads the public chain, and answers **"Unchanged ✓ · Sealed for: <username> · confirmed <date> · block N"** or **"Does not match ✗: this report was changed after it was sealed"**. (A *block* is one batch of chain entries, with a time.)

Why it matters for advice: an adviser can show that what it told a client has not been edited since, and nobody has to publish anything about the client to show it.

2. What goes where

Where	What is there	Who can read it
The public chain (Sepolia, chain id 11155111)	`EGOQ1` plus `SHA-256(salt ‖ record)`: 37 bytes (the label `EGOQ1`, 5 bytes, plus the 32-byte fingerprint) in a transaction that moves no money (0 ETH), from EgoQuant's sealing address (`0x3c9E28032A834F958BaBE168aa9AcC116D2c9ddC`) to itself	everyone; it reveals nothing about the record without the record and its salt (it does show that a seal was made, when, and from which address)
The database (Supabase; other users cannot read it: row-level security)	the record, its random salt and the report exactly as it was sealed, byte for byte	the account's owner (and the service that runs it)
The investor's download	the sealed report: the report, then its seal (the record, the salt and the transaction)	whoever holds the file

Words used here: the **salt** is a random secret number mixed in so that the fingerprint cannot be guessed; ‖ means "followed by"; SHA-256 is the standard fingerprint function. The record holds: the risk category, the questionnaire answers, the portfolio weights to the millionth, the Risk Engine's checks, the code and model versions, the steps the person took, the fingerprint of the sealed report, the report number and the **username the person chose**. It never holds an e-mail address or a user id.

3. What it proves, and what it does not

It proves that this exact report existed, unchanged, no later than that block's time, sealed for that username. Two attacks are addressed. A genuine seal shown next to an *edited* report is caught: the page computes the fingerprint of the report itself, so one changed character anywhere fails. A genuine report

shown by someone it was not sealed for is made *visible*: its first line says whom it was sealed for, and changing the name breaks the fingerprint.

It does not prove: - that the advice was right (it proves what was said, not that it was wise); - who is holding the file. A signed-in person can compare a transaction with **Account, My seals**, but the Verify page does not yet check "this report belongs to my account" automatically (decision OC-15); - anything about a printed or PDF copy: it cannot prove itself, only the sealed .html file or the technical seal file can be checked; - that the market data behind the numbers was right, or that the seal is a legal timestamp (it is a public record on a test network, not a notary); - permanence. Sepolia is a test network and makes no promise to keep its entries; a lasting record would be on a chain run by a group of institutions (section 8).

A check that does not use our website. The Verify page runs code that our own website serves, so a compromised website could print "Unchanged". A check that needs nothing of ours but the file: recompute the fingerprint with Python's standard library, `python3 -c "import json,hashlib,sys;p=json.load(open(sys.argv[1],encoding='utf-8'));print(hashlib.sha256(bytes.fromhex(p['salt'])+p['record'].encode()).hexdigest())" <seal file>`, and compare it with the transaction's input data on Etherscan (`0x45474f5131` + that value); the submission package's `6_Execution_Evidence/b_onchain/README.md` has this and the sealed-report version. Our own command-line verifier, `python -m egoquant.anchoring verify <file>` (`tests/anchoring/test_proof_cli.py`), runs the full set of checks but starts from the server's configuration, so it is a tool for the team rather than for an outside reader.

4. Check it yourself

A. Two minutes, no account. Sidebar, **Verify a report**. Press **Use the demo report**: a real report of an invented investor, sealed on Sepolia, is checked against the live chain (block 11,951,549) and shown below the verdict. Press **View on Etherscan** to see the transaction. Press **Try a tampered copy**: the verdict turns to "Does not match".

B. Five minutes, with an account (one account per person; `docs/JUDGE_GUIDE.md`). Build a portfolio while signed in, wait until the news rating of the report is finished, press **Confirm and seal**, read the text and agree. Usually within a minute the badge reads "Sealed on Sepolia ✓ · block N". **Download sealed report**, open **Verify a report**, upload the file: "Unchanged ✓ · Sealed for: <you>". Change any number in the file with a text editor and upload it again: "Does not match".

C. Without our website. Recompute the fingerprint with the one line of section 3 and compare it with the transaction, or open the transaction on Etherscan: its input data must start with `0x45474f5131` (the letters EGOQ1) followed by the 32-byte fingerprint, and it must come from the sealing address above.

5. Privacy and deletion

- Before the first seal the person reads and accepts a consent text (version `2026-10-20-v2`). It says what is kept, that only a fingerprint goes on the chain, and that a public chain entry can never be deleted.
- **"Delete my data"** removes the record, its salt and the sealed report (one database row; `supabase/migrations/007_erase_everything.sql`, `supabase/migrations/008_seal_v2.sql`), so we can no longer link the fingerprint to a person from our database (the database provider's backups age out under its own rules). The chain entry itself stays. Your sign-in and username stay (deleting the account removes them). Someone who kept a downloaded sealed report still holds its record and salt.
- Sealing never blocks advice: when the chain or its provider is down the plan is still confirmed and the badge says "Sealing… (the fingerprint will be sent shortly; your plan is confirmed)".

6. Safety rules

- **Test network only.** Any chain id other than 11155111 is refused.
- **The signing key is kept in Railway's environment** (`EGOQ_SEAL_PRIVATE_KEY`): it is not in the repository, and no public endpoint returns it (`tests/test_no_secrets_in_public_endpoints.py`).
- **The tests never use the network or a real key** (a fake chain); one marked-live test (`tests/anchoring/test_live_seal.py`, skipped without the key, never run by the automatic test run) makes a real seal.
- **Limits:** 5 seals per account per day, 200 a day in total (provisional numbers; past them the plan is still confirmed and the seal waits).
- **Key custody.** The key signs only 0-ETH self-transactions on a test network; it holds test ETH and nothing else. If it ever had to be replaced, the old address goes into `EGOQ_SEAL_PUBLISHED_ADDRESSES` and every older seal still verifies.
- **One server, one transaction at a time.** The service runs as one server (one Railway replica) and sends one transaction at a time, numbered in order, so, in normal running, seals made at the same moment never replace each other (during a redeploy the old and the new server can overlap for a moment; `tests/anchoring/test_client.py::test_seals_sent_at_the_same_moment_get_different_nonces`). Each seal costs at most 21,592 gas (units of transaction fee).
- **When the chain provider is down,** the plan is still confirmed and the seal waits (`tests/anchoring/test_routes.py::test_rpc_down_still_confirms_then_recovers`); a pending seal is sent again with the **same** fingerprint (`python -m egoquant.anchoring retry`; `tests/anchoring/test_proof_cli.py::test_cli_rpc_down_then_retry_uses_the_same_commitment`).
- **Older seals keep verifying.** Seal version 2 is additive (`egoq-record-2`, `egoq-proof-2`); version 1 files verify with the note "older seal format, no document fingerprint".

7. Evidence

Claim	Where it is shown
Five real seals of real engine output for invented investors (A1 to A5) and five hand-written test seals (B1 to B5), with Etherscan links and tampered-copy results	`docs/ON_CHAIN_EVIDENCE.md`
The Verify page's own demo report is a real sealed report, checked against the live chain on 20 Oct	screenshot `docs/judges/screens/10-verify-proof-desktop.png`
Every proof committed as evidence still verifies with the final code, every tampered copy still fails, and the files are byte-exact (no line-ending rewrite)	`tests/anchoring/test_committed_proofs.py` (against a read-only snapshot of 11 real Sepolia transactions: the ten above plus the Verify page's demo report)
One changed character, a changed username, a forged footer, an older seal, the My seals list, the missing-migration fallback	`tests/anchoring/test_seal_v2.py`
The record, the commitment, the chain client, every mismatch kind and the command line	`tests/anchoring/` (465 tests, 464 run by default; one live test needs the key)
The browser code and the backend agree	the frontend repository's vector check (323 of 323 checks, including a check that the sealed demo files have no Windows line endings)

8. Roadmap

The same commitment on a chain run by a group of institutions (a consortium chain), such as WeBank's FISCO BCOS: what would change is the chain client and the chain id; what stays is the record, the salt, the commitment and the verifier (decision OC-19, not built).

Hypothetical, based on past data; not a forecast, not a recommendation. EgoQuant places no orders and moves no money.